VERIQTA

- LEARN
- BUILD
- DEPLOY
- OPERATE

- ✓ Reliable
- ✓ Scalable
- ✓ Secure
- ✓ Production Ready

RabbitMQ™ PRODUCTION HANDBOOK

From Messaging Fundamentals to Reliable Production Operations

Asynchronous Communication | Decouple Applications | Scale Reliably | Handle Failures | Run in Production | Real World Use Cases

MESSAGES MOVE BUSINESS.
LEARN RABBITMQ. BUILD RELIABLE SYSTEMS.

Better Engineers
A Stronger Tomorrow

@veriqta | veriqta | veriqta | veriqta | @veriqta | @veriqta

VERIQTA

Learn Build Deploy Grow

RABBITMQ EXPLAINED,
WHY APPLICATIONS NEED MESSAGING

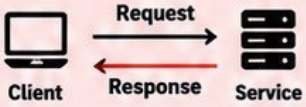
Asynchronous Communication for Scalable, Reliable and Resilient Systems

1 THE PROBLEM RABBITMQ SOLVES

- Modern applications have many moving parts and services.
- Direct communication between services can be slow, unreliable and hard to scale.
- If a service is down or overloaded, requests can fail.
- RabbitMQ solves this by decoupling services and buffering messages until they can be processed.

2 SYNCHRONOUS vs ASYNCHRONOUS COMMUNICATION

SYNCHRONOUS



- Client waits for a response.
- Slower and can fail if service is down.
- Works well for simple, real-time requests.

ASYNCHRONOUS

- Client sends a message and continues.
- Service processes the message later.
- More reliable, scalable and resilient.

3 WHAT A MESSAGE BROKER IS

A message broker is software that receives messages from producers, stores them safely, and delivers them to consumers when they are ready.
RabbitMQ is a message broker that uses the AMQP protocol.

4 PRODUCER → RABBITMQ → CONSUMER

5 DECOUPLING APPLICATIONS

- Services do not need to know about each other.
- They communicate through RabbitMQ.
- Each service can be developed, deployed and scaled independently.
- Failures in one service do not immediately affect other services.

6 BUFFERING WORK DURING TRAFFIC SPIKES

7 RABBITMQ vs DIRECT API CALLS

DIRECT API CALLS	RABBITMQ (MESSAGE BROKER)
✗ Services are tightly coupled	✓ Services are decoupled
✗ Requests can fail if service is down	✓ Messages are stored and retried
✗ Harder to handle traffic spikes	✓ Handles traffic spikes
✗ No built-in message persistence	✓ Supports durable message storage
✗ Less flexible for retries and recovery	✓ More flexible and resilient

8 REAL DEVOPS USE CASES

- E-commerce order processing
- Log and event processing
- Sending emails and notifications
- Background jobs (reports, data processing)
- Data synchronization between services
- Streaming data and analytics pipelines

9 JUNIOR ENGINEER TIP

RabbitMQ is infrastructure, not just an application library.
As a DevOps or platform engineer, you need to understand how to deploy, configure, monitor and operate RabbitMQ in production, not just how to use it in code.

Messaging Builds Better Systems

Learn Build Deploy Grow

RABBITMQ ARCHITECTURE AND THE MESSAGE JOURNEY

From Producers to Consumers, Understanding How Messages Flow

1 KEY COMPONENTS

Producer

- Sends messages to RabbitMQ.
- Can be an application, service or system.

Connection

- A TCP connection between your application and RabbitMQ.
- Can have multiple channels.

Channel

- A lightweight communication channel inside a connection.
- Used to publish, consume and manage messages.

Exchange

- Receives messages from producers.
- Routes messages to queues based on rules (routing key, headers, etc).

Binding

- A rule that connects an exchange to a queue.
- Defines how messages should be routed.

Queue

- Stores messages until a consumer retrieves them.
- Can be durable, exclusive or auto-delete.

Consumer

- Receives messages from a queue.
- Processes the messages and acknowledges them.

Virtual host (vhost)

- A logical separation inside RabbitMQ.
- Allows multiple applications/teams to use the same cluster securely.

2 THE MESSAGE JOURNEY

Producer
Publishes a message with a routing key

→

Exchange
Receives the message and uses routing rules

→

Binding
Connects the exchange to the queue

→

Queue
Stores the message until a consumer retrieves it

→

Consumer
Consumes the message and processes it (acknowledges when done)

3 MESSAGE LIFECYCLE

1. Producer creates a message.
2. Message is sent to an exchange with a routing key.
3. Exchange routes the message to one or more queues (via bindings).
4. Message stays in the queue until a consumer requests it.
5. Consumer receives the message.
6. Consumer processes the message.
7. Consumer acknowledges (ack) the message.
8. Message is removed from the queue.

THE MENTAL MODEL

Producer → Exchange → Binding → Queue → Consumer

This simple flow helps you understand how messages move through RabbitMQ.

4 WHAT HAPPENS WHEN A COMPONENT FAILS?

Component	What Happens
Producer	Messages are not published. Downstream consumers receive nothing.
Connection	The producer or consumer loses connection. Automatic reconnection is usually required.
Channel	Publishing or consuming stops on that channel. Other channels may continue to work.
Exchange	Messages cannot be routed. Publishers may get errors or messages may be lost (if not using confirms).
Binding	Messages will not reach the intended queue. They may be routed elsewhere or dropped.
Queue	Messages cannot be stored. If the queue is deleted, messages may be lost (unless mirrored/ quorum).
Consumer	Messages remain in the queue. They will be delivered to another consumer if available.
Virtual host	Applications using that vhost cannot publish or consume.

JUNIOR ENGINEER TIP

RabbitMQ is infrastructure, not just an application library.

You need to understand how it works, how to operate it, monitor it, and troubleshoot it in production.

Learn Build Deploy Grow

BUILD YOUR FIRST RABBITMQ ENVIRONMENT

A Hands-On Lab to Run, Explore and Use RabbitMQ

1 RUNNING RABBITMQ LOCALLY

You can run RabbitMQ locally using Docker. This gives you a safe environment to learn and practice without installing anything on your system.

2 DOCKER-BASED LAB

```
# Run RabbitMQ with management plugin
docker run -d --name rabbitmq \
  -p 5672:5672 \
  -p 15672:15672 \
  rabbitmq:3-management
```

3 MANAGEMENT PLUGIN

- The management plugin provides a web-based UI to monitor and manage RabbitMQ.
- It is included in the rabbitmq:3-management image.
- You can also enable it manually using rabbitmq-plugins.

4 MANAGEMENT UI

Access the UI at:
http://localhost:15672

Default login (do not use in production):
Username: guest
Password: guest

5 IMPORTANT PORTS

AMQP Port (for applications)
5672 Used by your applications (producers and consumers).

Management Port (for UI)
15672 Used for the web management UI and HTTP API.

6 USEFUL COMMANDS

```
# Check server status
rabbitmqctl status
# List plugins
rabbitmq-plugins list
# Enable management plugin (if needed)
rabbitmq-plugins enable rabbitmq_management
# Show cluster info
rabbitmq-diagnostics cluster_status
# Check listeners (ports)
rabbitmq-diagnostics listeners
```

7 CREATING A TEST USER

```
# Create a new user
rabbitmqctl add_user devuser devpassword
# Set user permissions (.* for full access in this vhost)
rabbitmqctl set_permissions -p / devuser ".*" ".*" ".*"
# List users
rabbitmqctl list_users
```

8 CONNECTING YOUR FIRST PRODUCER AND CONSUMER

- You can use any language (Python, Java, Node.js, Go, etc.).
- Install a RabbitMQ client library.
- Connect to localhost on port 5672.
- Publish a test message from a producer.
- Consume the message from a consumer.
- Verify the message is received in the Management UI.

9 SECURITY WARNING

- The default guest/guest credentials are for local development only.
- Do not use default credentials in production.
- Create your own users with least privilege permissions.
- Restrict network access to the management UI.
- Use strong passwords and consider TLS in non-local environments.

> Never expose the RabbitMQ management UI to the internet in production.

Learn Build Deploy Grow

QUEUES EXPLAINED, WHERE MESSAGES WAIT

Understand How Queues Work and Keep Your Applications Reliable

1 WHAT A QUEUE ACTUALLY DOES

- A queue stores messages until a consumer is ready to process them.
- It acts as a buffer between producers and consumers.
- Queues help you handle spikes, retries and slow consumers.
- Messages stay in the queue even if the consumer is temporarily unavailable (depends on durability settings).

2 QUEUE NAMES

- Queues have unique names within a virtual host.
- Use clear and meaningful names (e.g. orders, email-tasks, logs).
- Avoid special characters and keep names consistent across environments.

Good: orders, payment-events, user-notifications

Avoid: Orders!, user notifications, test_queue_1

3 DURABLE QUEUES

- Survive broker restarts (when used with persistent messages).
- Defined with: durable: true
- Recommended for production.

4 EXCLUSIVE QUEUES

- Can only be used by the connection that declared them.
- Automatically deleted when the connection is closed.
- Useful for temporary, private queues (e.g. RPC).
- Not suitable for shared workloads.

5 AUTO-DELETE QUEUES

- Automatically deleted when the last consumer disconnects.
- Useful for temporary queues.
- Defined with: auto_delete: true
- Commonly used in development and testing.

6 QUEUE ARGUMENTS

Queue arguments allow you to configure additional behavior.

Examples:

- x-message-ttl (message time to live)
- x-max-length (limit queue size)
- x-dead-letter-exchange (DLX for failed messages)
- x-dead-letter-routing-key (routing key for DLX)

7 READY vs UNACKNOWLEDGED MESSAGES

Ready Messages

- Messages in the queue waiting to be consumed.
- Visible in the management UI as "Ready".
- Not yet delivered to a consumer.

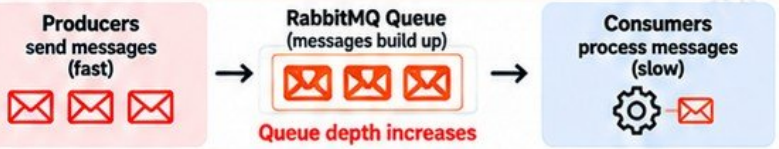
Unacknowledged Messages

- Messages that have been delivered to a consumer but not yet acknowledged.
- Visible in the management UI as "Unacked".
- Will be requeued if the consumer fails or disconnects (if using manual ack).

8 QUEUE DEPTH

- Queue depth is the total number of messages in the queue (Ready + Unacked).
- A growing queue depth can indicate:
  - Consumers are slower than producers
  - Consumers are down or failing
  - More traffic than expected
- Monitor queue depth to detect problems early.

9 WHEN CONSUMERS ARE SLOWER THAN PRODUCERS



If producers send messages faster than consumers can process them, the queue will keep growing. This can consume disk space and memory, and may eventually cause performance issues.

10 BEGINNER MISTAKE

Treating RabbitMQ as unlimited storage.

- Queues are not a database.
- They have limits (memory, disk, TTL, max length).
- If consumers do not keep up, the queue can grow and cause issues (disk full, performance problems, messages rejected).
- Always monitor queue depth and set appropriate limits.

Design your system so messages are processed in a timely manner. Use dead-letter queues, retry strategies and monitoring to handle backlogs.

Learn Build Deploy Grow

EXCHANGES AND ROUTING, HOW RABBITMQ FINDS THE QUEUE

Understand Routing Strategies and Choose the Right Exchange for Your Use Case

1 WHY PRODUCERS NORMALLY PUBLISH TO EXCHANGES

- Producers publish to exchanges instead of queues to decouple message producers from consumers.
- Exchanges use routing rules to send messages to one or more queues.
- This allows flexible, scalable and maintainable architectures.

2 ROUTING KEYS

- A routing key is a string attached to a message by the producer.
- The exchange uses the routing key to decide which queue(s) should receive the message (based on bindings and rules).
- Not all exchange types use routing keys (e.g., fanout ignores routing keys).

3 BINDINGS

- Bindings connect an exchange to a queue.
- They define the routing rules (e.g., routing key, pattern or headers).
- A queue can be bound to an exchange using one or more binding rules.
- A message is delivered to a queue if it matches the binding rule.

4 CHOOSING THE CORRECT EXCHANGE TYPE

- Use **Direct** when you need exact routing based on a routing key.
- Use **Fanout** when you want to send to all bound queues.
- Use **Topic** when you need pattern-based routing.
- Use **Headers** when you need routing based on message headers.
- Use **Default** for simple, direct-to-queue scenarios or for testing.

5 DIRECT EXCHANGE

- Routes messages to queues with an exact matching routing key.
- Best for simple, explicit routing.

Use case:
- Order processing
- Task queues

6 FANOUT EXCHANGE

- Routes messages to all bound queues (ignores routing key).
- Best for broadcasting messages.

Use case:
- Logs and monitoring
- Real-time notifications

7 TOPIC EXCHANGE

- Routes messages based on routing key patterns (wildcards).
- Supports * (one word) and # (zero or more words).

Use case:
- Event-driven architectures
- Multi-service systems

8 HEADERS EXCHANGE

- Routes messages based on message headers (instead of routing key).
- Best for complex routing rules.

Use case:
- Enterprise systems
- Routing based on metadata (e.g., type, region)

9 DEFAULT EXCHANGE

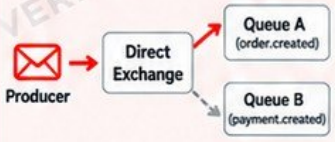
- Built-in exchange with an empty name ("").
- Routes messages to a queue with the same name as the routing key.
- Useful for simple scenarios and testing.

Use case:
- Quick tests
- Direct queue communication

10 ROUTING BEHAVIOR COMPARISON

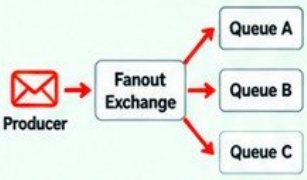
Direct Exchange

routing key = order.created



Only queues with an exact matching routing key receive the message.

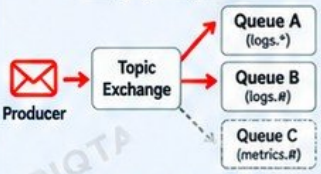
Fanout Exchange



Sends the message to all bound queues (ignores routing key).

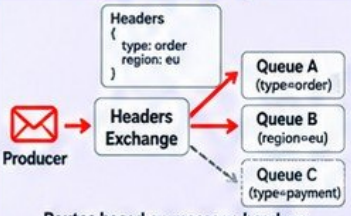
Topic Exchange

routing key = logs.error



Routes based on pattern matching (wildcards).

Headers Exchange



Routes based on message headers (key-value pairs).

KEY TAKEAWAY

Exchanges give you flexibility. Choose the right exchange type based on your routing requirements and design your bindings carefully.

COMMON MISTAKE

Using the wrong exchange type can lead to messages not being delivered to the right queues.

CONNECTIONS AND CHANNELS IN PRODUCTION

Learn Build Deploy Grow

Understand How Clients Connect, Communicate and Stay Reliable at Scale

1 TCP CONNECTIONS

- A TCP connection is a long-lived network connection between your application and RabbitMQ.
- All AMQP communication happens over this connection.
- Typically uses port 5672 (or 5671 for TLS).

2 AMQP CHANNELS

- A channel is a lightweight virtual connection inside a TCP connection.
- Channels allow multiple independent streams of communication.
- Each channel has its own channel ID.

3 WHY CHANNELS EXIST

- Channels let you perform multiple operations in parallel without opening multiple TCP connections.
- More efficient use of resources.
- If a channel closes due to an error, the TCP connection can remain open and other channels continue to work.

4 CONNECTION REUSE

- Create a small number of long-lived connections and reuse them.
- Do not open a new connection for every message.
- Connection reuse reduces overhead and improves performance.
- Recommended: one connection per application instance (or per thread/process), with multiple channels.

5 CHANNEL LIFECYCLE

- Channels are created after a connection is established.
- Use channels for publishing, consuming and management operations.
- Close channels when they are no longer needed.
- A channel can be closed explicitly or automatically (if an error occurs).

6 HEARTBEATS

- Heartbeats keep the connection alive and detect network failures.
- If heartbeats are missed, the connection is closed.
- Helps detect broken connections early.
- Recommended heartbeat value is usually 5–60 seconds (depending on your environment).

7 CONNECTION RECOVERY

- If a connection is lost, clients can automatically reconnect.
- After reconnection, channels need to be recreated and topology redeclared (exchanges, queues, bindings).
- Use client libraries that support automatic recovery (e.g., Java, Python, .NET, Node.js clients).

8 CONNECTION LEAKS

- Connection leaks happen when connections are not closed properly.
- Too many open connections can overload RabbitMQ.
- Symptoms: increasing connection count, high memory usage.
- Always close connections and channels when your application shuts down.

9 CHANNEL EXHAUSTION

- Each connection has a limit on the number of channels (default is 2047).
- Opening too many channels can cause channel exhaustion errors.
- Reuse channels instead of creating new ones unnecessarily.
- Monitor channel usage in production.

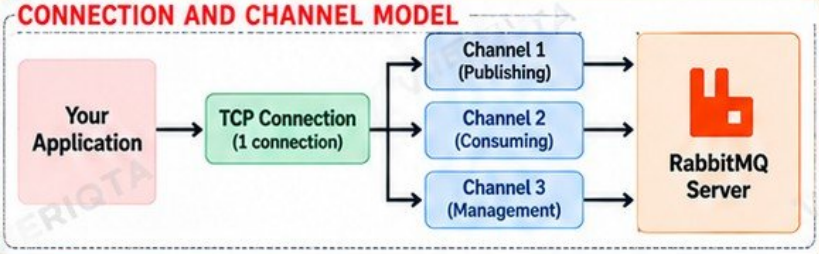
10 WHY OPENING A CONNECTION FOR EVERY MESSAGE IS A BAD DESIGN

- Creates high overhead (TCP handshake, authentication, setup).
- Consumes server resources quickly.
- Leads to connection leaks and can crash your application or RabbitMQ.
- Does not scale and increases latency.
- Use long-lived connections and multiple channels instead.

11 PRODUCTION CONNECTION PATTERN

- Create a small number of long-lived connections.
- Use multiple channels per connection.
- Enable heartbeats.
- Use automatic connection recovery.
- Monitor connection and channel counts.
- Follow your client library's best practices.

CONNECTION AND CHANNEL MODEL



KEY TAKEAWAYS

- Use few long-lived connections.
- Use multiple channels.
- Enable heartbeats and automatic recovery.
- Avoid connection leaks and channel exhaustion.
- Do not open a new connection for every message.
- Design for reliability and scalability.

PRODUCERS, PUBLISHING MESSAGES RELIABLY

Learn Build Deploy Grow

Send Messages Correctly, Safely and Ensure They Reach the Queue

1 PUBLISHING WORKFLOW

- Create a connection to RabbitMQ.
- Open a channel.
- Publish a message to an exchange with a routing key.
- Set message properties (e.g., content type, headers, persistence).
- Use publisher confirms to ensure delivery.

2 EXCHANGE AND ROUTING KEY

- Producers publish to an exchange, not directly to a queue.
- Specify the exchange name and routing key.
- The exchange uses the routing key and bindings to route the message to the correct queue(s).

3 MESSAGE PROPERTIES

- Message properties provide metadata about the message.
- Common properties: content type, message ID, headers, expiration time, etc.
- They help with routing, processing and debugging.

4 CONTENT TYPE

- Specifies the format of the message body.
- Common values: application/json, application/xml, text/plain.
- Consumers use this to understand how to deserialize the message.

5 MESSAGE IDs

- A unique ID for each message (e.g., UUID).
- Helps with tracking, deduplication and troubleshooting.
- Useful when retrying or handling failures.

6 PERSISTENT MESSAGES

- Mark messages as persistent so they survive broker restarts (when used with a durable queue and exchange).
- Set the delivery mode to 2 (persistent).
- Non-persistent messages may be lost if the broker restarts.

7 MANDATORY PUBLISHING

- Use the mandatory flag to ensure the broker returns unroutable messages.
- If a message cannot be routed to any queue, it will be returned to the producer.
- Helps prevent silent message loss.

8 UNROUTABLE MESSAGES

- Messages may be unroutable if there is no matching binding for the routing key.
- With mandatory publishing, the message is returned to the producer.
- Without it, the message is discarded.
- Always handle returned messages.

9 PUBLISHER CONFIRMS

- Publisher confirms tell you whether the broker has successfully received and stored the message.
- Enables reliable publishing.
- Use confirm mode on the channel and wait for ack or nack.

10 CONFIRM FAILURES

- A nack means the broker failed to store the message.
- This can happen due to disk issues, node failures or internal errors.
- Handle nacks and implement a retry strategy.

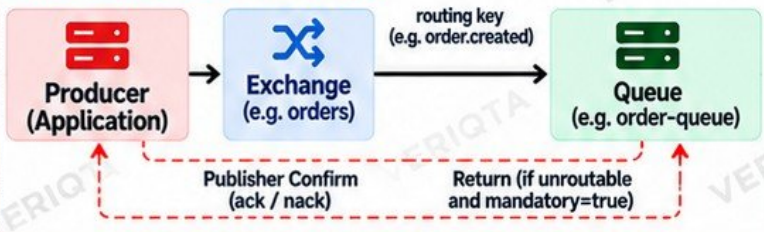
11 RETRYING SAFELY

- Implement retries with exponential backoff.
- Avoid immediate retries to prevent overwhelming the broker.
- Use a maximum retry limit.
- Log failed messages for inspection or send them to a dead-letter queue.

12 WHY "PUBLISH SUCCEEDED" DOES NOT ALWAYS MEAN "MESSAGE IS SAFE"

- A successful publish call only means the message was accepted by the server, not that it was stored.
- Without publisher confirms, the message can still be lost due to broker failures.
- Always use persistent messages, durable queues and publisher confirms for reliable delivery.

PRODUCER PUBLISHING FLOW



KEY TAKEAWAYS

- Always publish to an exchange.
- Set message properties (content type, message ID).
- Use persistent messages with durable queues.
- Enable mandatory publishing.
- Use publisher confirms.
- Handle failures and retry safely.
- Do not assume a successful publish means the message is stored.

CONSUMERS,

ACKNOWLEDGEMENTS AND SAFE PROCESSING

Learn Build Deploy Grow

Process messages reliably without losing or duplicating work

1 CONSUMER LIFECYCLE

A consumer connects to RabbitMQ, subscribes to a queue, receives messages and processes them.

2 AUTOMATIC vs MANUAL ACKNOWLEDGEMENT

AUTOMATIC ACK (auto-ack)

- Message is marked as acknowledged immediately after it is delivered.
- Simple to implement.
- If your application crashes during processing, the message is lost.
- Not recommended for production.

MANUAL ACK (recommended)

- You explicitly acknowledge after successful processing.
- Safer and more reliable.
- If your application crashes, the message is returned to the queue.
- Used in production applications.

3 ACK, NACK AND REJECT

Action	What It Does	Common Use Case
ack	Confirms the message was processed successfully.	Use after your application finishes processing without errors.
nack	Rejects the message. Can requeue or discard it.	Use when processing fails. You can choose to requeue the message.
reject	Rejects the message. (No requeue option in basic.reject)	Use when the message should not be reprocessed (e.g. invalid data).

4 REQUEUE BEHAVIOR

REQUEUE = TRUE

- Message is returned to the queue.
- Can be picked up by the same or another consumer.
- Useful for temporary failures.
- Use with caution to avoid retry loops.

REQUEUE = FALSE

- Message is discarded (or sent to a dead letter exchange if configured).
- Use when the message cannot be processed (e.g. invalid data).
- Helps prevent endless retry cycles.

5 CONSUMER CRASHES

- If a consumer crashes before acknowledging:
- The message remains unacknowledged.
- RabbitMQ will return the message to the queue.
- Another consumer can then process it.
- This is why manual acknowledgements are important for reliability.

6 UNACKNOWLEDGED MESSAGES

- Unacknowledged messages are messages that have been delivered but not yet acknowledged.
- They are invisible to other consumers.
- If the consumer disconnects or crashes, these messages return to the queue.
- Monitor unacknowledged messages to detect stuck or slow consumers.

7 PREFETCH AND FAIR DISPATCH

- Prefetch limits the number of unacknowledged messages a consumer can receive.
- Example: prefetch = 1 → a consumer gets one message at a time.
- Helps distribute messages fairly across multiple consumers (fair dispatch).
- Prevents one consumer from being overloaded.

8 SLOW CONSUMERS

- A slow consumer takes longer to process messages.
- Messages will build up in the queue.
- Other consumers can help if prefetch is set correctly.
- Monitor queue depth and consumer performance.
- Scale consumers horizontally when needed.

9 JUNIOR ENGINEER TIP

Acknowledge the message only after successful processing.
This ensures messages are not lost if your application fails, and it allows safe retry and recovery. RabbitMQ is infrastructure, so design your consumers to be reliable, resilient and idempotent.

Reliable Messaging Builds Reliable Systems

MESSAGE DURABILITY, WHAT SURVIVES A FAILURE?

Learn Build Deploy Grow

Keep your messages safe, even when things go wrong

1 DURABLE QUEUES

- A durable queue survives a RabbitMQ node restart.
- Queue metadata is saved to disk.
- Messages in a durable queue can survive restarts (if they are also persistent).
- Non-durable queues are deleted when the node restarts.

```
# Create a durable queue
queue.declare(queue='my-queue', durable=True)
```

2 PERSISTENT MESSAGES

- Persistent messages are written to disk (not just memory).
- They survive a node restart if the queue is durable.
- Non-persistent messages can be lost if the node crashes.

```
# Publish a persistent message
channel.basic_publish(
  exchange='', routing_key='my-queue',
  body='Hello', properties=BasicProperties(
    delivery_mode=2 # make message persistent
  )
)
```

3 DURABLE EXCHANGES

- A durable exchange survives a node restart.
- Exchange definitions are saved to disk.
- Non-durable exchanges are lost on restart.

```
# Create a durable exchange
channel.exchange_declare(
  exchange='my-exchange',
  exchange_type='direct',
  durable=True
)
```

4 PUBLISHER CONFIRMS

- Publisher confirms ensure the message has been safely written to disk on the broker.
- Without confirms, a message can be lost even if the publish call succeeds.
- Use confirms in production for reliability.

```
# Enable publisher confirms
channel.confirm_delivery()
# Publish and check for confirmation
channel.basic_publish(...)
# Wait for confirmation (library specific)
```

5 NODE RESTART SCENARIOS

- Durable queues, durable exchanges and persistent messages survive a node restart.
- Non-durable queues, exchanges and non-persistent messages are lost.
- If you have a cluster, surviving nodes keep your data available.
- Always test recovery scenarios in a non-production environment.

6 MEMORY vs DISK

MEMORY	DISK
Faster	Slower
Used for non-persistent messages	Used for persistent messages and metadata
Lost if the node crashes	Survives node restarts
Temporary storage	Required for durability

7 WHAT DURABILITY DOES NOT GUARANTEE

- It does not guarantee that a message will be processed.
- It does not prevent duplicate messages.
- It does not protect against application bugs.
- It does not replace the need for consumer acknowledgements.
- It does not guarantee availability if all cluster nodes are lost.
- It does not protect against misconfiguration.

8 RELIABILITY CHAIN

Persistent Message → Durable Exchange → Durable Queue → Publisher Confirms → Consumer Ack

ALL PARTS WORK TOGETHER

A weak link in this chain can still lead to message loss.

9 PRODUCTION AWARENESS

Durability requires more than one setting.

For real message safety, you need durable exchanges, durable queues, persistent messages, publisher confirms and proper consumer acknowledgements. Configure, test and monitor all of them.

Reliable Messages Power Real Systems

DELIVERY GUARANTEES AND DUPLICATE MESSAGES

Learn Build Deploy Grow

Understand delivery semantics and design resilient consumers

1 AT-MOST-ONCE

- Message is delivered zero or one time.
- If a failure happens, the message can be lost.
- No retries.
- Fast, but not reliable.
- Usually not suitable for critical workloads.

RISK:
Messages can be lost if the consumer or network fails.

2 AT-LEAST-ONCE

- Message is delivered one or more times.
- If a failure happens, the message is retried.
- May result in duplicate messages.
- More reliable and commonly used in production.

TRADE-OFF:
You must handle duplicate messages in your application.

3 WHY DUPLICATES HAPPEN

- Consumer crashes after processing but before ack.
- Network issues.
- Connection loss and recovery.
- Message redelivery by RabbitMQ.
- Retries from the consumer.
- Publisher retries.
- Multiple consumers processing the same message.
- Exactly-once is very hard to achieve in distributed systems.

4 REDELIVERY

- If a message is not acknowledged, it will be redelivered.
- Redelivered messages have a redelivered flag.
- You can check this flag in your consumer.
- Redelivery can happen multiple times.

```
# Check if a message was redelivered
if properties.redelivered:
  print("This is a redelivered message")
```

5 CONSUMER RETRIES

- Consumers can retry processing when a failure happens.
- Use retry limits to avoid infinite loops.
- Combine with backoff delays.
- After max retries, send the message to a dead-letter queue (DLQ).

6 IDEMPOTENCY

- Design your consumer so it can safely process the same message multiple times.
- Use unique identifiers to detect duplicates.
- Ensure the operation can be repeated without creating unwanted side effects (e.g. duplicate payments, duplicate records).

7 MESSAGE IDS

- Use a unique message ID (e.g. UUID) when publishing.
- Consumers can use the message ID to detect and ignore duplicates.
- Message ID can be stored in a database or cache.
- Helps with idempotency and deduplication.

8 DEDUPLICATION STRATEGIES

- Use message IDs with a database or cache (e.g. Redis).
- Check if the message has already been processed.
- Store processed IDs with a TTL.
- Use idempotent operations (e.g. UPSERT instead of INSERT).
- For high throughput, consider a distributed cache.

9 WHY "EXACTLY ONCE" SHOULD NOT BE CASUALLY ASSUMED

- "Exactly once" is extremely difficult in distributed systems.
- It requires coordination across multiple components (network, broker, consumer, database).
- Even with advanced patterns, edge cases can still produce duplicates.
- Design for at-least-once and handle duplicates instead.

10 DESIGNING CONSUMERS THAT SAFELY PROCESS THE SAME MESSAGE AGAIN

- Make your consumer idempotent.
- Use message IDs to track processed messages.
- Handle retries gracefully.
- Avoid side effects that cannot be safely repeated.
- Test your consumer with duplicate messages.
- Monitor and alert on high redelivery rates.

KEY TAKEAWAY

Build your consumers to be resilient. Assume duplicate messages will happen, and design your application to handle them safely.

Better Engineers Build Better Systems

DEAD LETTER EXCHANGES AND FAILED MESSAGES

Learn Build Deploy Grow

Handle failures gracefully and keep your system reliable

1 WHAT DEAD LETTERING MEANS

- Dead lettering is a way to move messages that cannot be processed to a separate queue for later inspection.
- It prevents problematic messages from blocking your main queue.
- Helps keep your application healthy and running.

Dead lettering does not delete the message. It moves it to a dead-letter exchange (DLX).

2 DEAD LETTER EXCHANGE (DLX)

- A DLX is a regular RabbitMQ exchange.
- It receives messages that are rejected, expired or dropped.
- You configure a queue to send failed messages to a DLX.
- The DLX then routes those messages to a dead-letter queue (DLQ) using a routing key.

3 DEAD-LETTER ROUTING

- You define the dead-letter exchange (DLX) in the queue arguments.
- You can also define a dead-letter routing key.
- The DLX works like any other exchange (direct, topic, etc).
- Failed messages are routed to a dead-letter queue based on the routing key.

```
# Queue with dead-letter exchange
x-dead-letter-exchange: my-dlx
x-dead-letter-routing-key: failed
```

4 REJECTED MESSAGES

- When a consumer rejects a message (basic.reject or basic.nack with requeue=false), it can be sent to the DLX.
- Common for messages that cannot be processed.
- Example: invalid data, failed validation.

5 EXPIRED MESSAGES

- Messages can expire based on a TTL (time to live).
- When a message expires, it is routed to the DLX.
- Useful for time-sensitive messages.

```
# Message TTL (in milliseconds)
x-message-ttl: 60000
```

6 QUEUE LENGTH LIMITS

- You can set a maximum queue length.
- When the queue is full, new messages can be rejected and sent to the DLX.
- Helps prevent unlimited growth and protects your system.

```
# Max queue length
x-max-length: 10000
```

7 DEAD-LETTER QUEUES

- A DLQ stores failed messages for later inspection.
- You can have separate DLQs for different failure reasons.
- Messages remain in the DLQ until you manually process or delete them.

8 INSPECTING FAILED MESSAGES

- Use the RabbitMQ Management UI to view messages in the DLQ.
- Check message headers, routing key and reason.
- You can requeue, republish or export the message for analysis.

9 RETRY vs DEAD-LETTER

- Retry: try processing the message again (often after a delay).
- Dead-letter: move the message to a DLQ when it cannot be processed.
- Use retries for temporary failures and dead-lettering for permanent failures.

10 POISON MESSAGES

- A poison message always fails (e.g. invalid format, bad data).
- Repeated retries can waste resources and block the queue.
- Send poison messages to a DLQ after a limited number of retries.
- Manually inspect and fix the data before reprocessing.

11 FAILURE FLOW

Main Queue → Consumer Failure → DLX → Dead-Letter Queue

Better Engineers Build Better Systems

FAILED MESSAGES ARE NOT LOST, THEY ARE OPPORTUNITIES TO BUILD A MORE RESILIENT SYSTEM.

Reliable Messaging Stronger Systems

RETRIES WITHOUT CREATING A PRODUCTION DISASTER

Learn Build Deploy Grow

Retry smart. Keep your systems stable. Avoid making failures worse.

1 WHY IMMEDIATE RETRIES CAN MAKE OUTAGES WORSE

- If a downstream service is down, immediate retries add more load.
- This can increase latency, exhaust resources and make the outage last longer.
- Many consumers retry at the same time, creating a retry storm.
- Use delayed retries and backoff instead of retrying immediately.

Retrying too fast can turn a small issue into a full production outage.

2 RETRY QUEUES

- A retry queue holds failed messages for later processing.
- Use separate queues for retries instead of requeuing immediately to the main queue.
- This gives time for the underlying issue to be resolved.
- You can have multiple retry queues with different delays.

3 DELAYED RETRIES

- Delayed retries wait for a specific time before the message is retried.
- You can use TTL with dead-letter exchanges or delay plugins.
- Common delays: 30 seconds, 1 minute, 5 minutes, 15 minutes.
- Use increasing delays (exponential backoff).

4 TIME TO LIVE (TTL)

- TTL sets how long a message stays in a queue.
- When the TTL expires, the message is moved to a dead-letter exchange (DLX).
- TTL is commonly used to implement delayed retries.

```
# Example: message TTL (10 minutes)
x-message-ttl: 600000
```

5 RETRY LIMITS

- Always set a maximum number of retries.
- Use a retry counter in message headers or in your application.
- After the limit is reached, move the message to a dead-letter queue for manual inspection.

6 BACKOFF

- Use exponential backoff to increase the delay between retries.
- Example: 30s → 1m → 5m → 15m → 1h.
- Helps reduce load on failing services.
- Avoid retrying too many times in a short period.

Exponential backoff = Be kinder to your infrastructure.

7 POISON-MESSAGE LOOPS

- A poison message always fails (e.g. invalid data).
- If retried indefinitely, it can block the queue and waste resources.
- Use retry limits and dead-letter queues.
- Inspect and fix the message or handle it manually.

A poison message can keep failing forever. Stop the loop.

8 RETRY STORMS

- A retry storm happens when many consumers retry at the same time.
- It can overwhelm your system and other dependencies.
- Use delayed retries, backoff and jitter.
- Monitor retry rates and set alerts.

9 WHEN NOT TO RETRY

- Do not retry for permanent failures (e.g. validation errors).
- Do not retry poison messages repeatedly.
- Do not retry if the operation is not idempotent and can cause side effects.
- Do not retry indefinitely.
- Use dead-letter queues for messages that cannot succeed.

Not every failure should be retried. Some failures need human attention.

10 PARKING FAILED MESSAGES

- Move messages that have reached the retry limit to a parking queue (DLQ).
- Keep the message for manual inspection.
- Analyse, fix the data or reprocess it later.
- Helps keep your main queues healthy.

11 RETRY → INSPECT → RECOVER WORKFLOW

Message Fails → Retry with Delay (TTL) → Process Again → Inspect (if still failing) → Recover (Fix or Reprocess)

Retry → Inspect → Recover → Keep Your System Stable

Better Engineers Build Better Systems

Reliable Messaging Stronger Systems

QUEUE TYPES

CLASSIC vs QUORUM vs STREAMS

Choose the right queue type for your workload and reliability needs

Learn Build Deploy Grow

1 CLASSIC QUEUES

- The original RabbitMQ queue type.
- Stores messages on a single node (by default).
- Supports durability and mirroring (classic HA).
- Well suited for simple workloads and lower throughput.
- Many existing applications already use classic queues.

Best for:
General purpose workloads, small to medium systems.

2 QUORUM QUEUES

- Designed for high availability and data safety.
- Replicated using the Raft consensus algorithm.
- Messages are stored on multiple nodes (majority).
- Survives node failures without losing messages.
- Recommended for most production workloads.

Best for:
Critical workloads that require high reliability and fault tolerance.

3 STREAMS

- A high-throughput, log-based queue type.
- Optimized for large volumes of messages.
- Stores messages as an append-only log.
- Designed for long-term storage and fast replay.
- Supports multiple consumers and offset-based consumption.

Best for:
High-throughput workloads, event streaming and log data.

4 REPLICATION DIFFERENCES

Feature	Classic	Quorum	Streams
Replication Model	Single node (or mirrored)	Raft (majority)	Raft (log based)
Data Safety	Lower (by default)	High	High
Survives Node Failure	With mirroring	Yes (majority)	Yes (majority)
Recommended for Production	Only for simple use cases	Yes	Yes (for streaming)

5 RELIABILITY CHARACTERISTICS

Feature	Classic	Quorum	Streams
Message Durability	Yes	Yes	Yes
Data Loss Risk (on node failure)	Higher (unless mirrored)	Low	Low
Fault Tolerance	Limited	High	High
Designed For	Basic reliability	Strong reliability	High reliability at scale

6 ORDERING CONSIDERATIONS

Classic
- Maintains order per queue.
- Ordering can be affected during failover (mirrored).

Quorum
- Maintains order per queue.
- Stronger ordering guarantees.

Streams
- Maintains order using a log.
- Consumers read by offsets.
- Best for time-ordered event data.

7 THROUGHPUT CONSIDERATIONS

Classic
- Good for low to moderate throughput.
- Can become a bottleneck at high scale.

Quorum
- Higher overhead due to replication.
- Optimized for reliability, not maximum throughput.

Streams
- Designed for very high throughput (millions of messages).
- Ideal for event streaming and analytics pipelines.

8 STORAGE CHARACTERISTICS

Classic
- Stores messages in traditional queue structure.
- Uses more disk I/O under heavy load.

Quorum
- Stores messages using a replicated log.
- More efficient and safer storage.

Streams
- Append-only log storage.
- Optimized for large volumes.
- Supports long-term retention and replay.

9 CHOOSING BASED ON WORKLOAD

Workload Type	Recommended Queue Type
Simple workload, low traffic	Classic
Critical apps (payments, orders, etc.)	Quorum
High-throughput event data (logs, events)	Streams
Need long-term storage and replay	Streams
Legacy application	Classic
New production system	Quorum (default)

10 PRODUCTION AWARENESS

- Different workloads have different requirements. There is no single "best" queue type.
- Selecting the right queue type improves reliability, performance and cost efficiency.
- Understand the trade-offs before going to production.
- You can use different queue types in the same RabbitMQ cluster.
- A well-designed architecture deliberately chooses the queue type based on business and technical requirements.

"The right queue type prevents problems today and gives you room to scale tomorrow."

Reliable Messaging Stronger Systems

Learn Build Deploy Grow

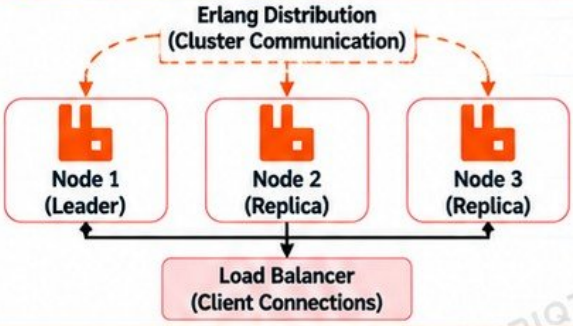
RABBITMQ CLUSTERING AND HIGH AVAILABILITY

Build Reliable Messaging Systems for Production

1 WHAT A RABBITMQ CLUSTER IS

- A cluster is a group of RabbitMQ nodes that work together as a single system.
- Nodes share metadata and can host queues, exchanges and bindings.
- Provides high availability, scalability and fault tolerance.
- Clients can connect to any node in the cluster.

2 CLUSTER ARCHITECTURE



Cluster Shares:

- Metadata
- Queue definitions
- User accounts
- Permissions
- Policies
- Virtual hosts
- Cluster state

3 NODES

- Each node runs a RabbitMQ instance.
- Nodes can be on the same server or different servers.
- Nodes communicate using Erlang distribution over a secure network.
- A cluster can have 2, 3 or more nodes.

4 ERLANG DISTRIBUTION

- RabbitMQ nodes communicate using Erlang distribution.
- Requires network connectivity and a shared cookie for authentication.
- Handles cluster membership, metadata synchronization and node discovery.

5 CLUSTER METADATA

- Information about queues, exchanges, bindings, users and permissions.
- Stored consistently across nodes.
- Allows any node to understand the cluster state.

6 QUEUE PLACEMENT

- Queues can be placed on specific nodes.
- Classic queues: live on a single node (unless mirrored, which is less common now).
- Quorum queues: replicated across multiple nodes (recommended for HA).
- Queue placement affects performance, availability and failover behavior.

7 QUORUM, LEADER AND REPLICAS

- Quorum queues use a Raft consensus model.
- One node is the leader (handles writes).
- Other nodes are replicas (keep copies).
- If the leader fails, a new leader is elected automatically.
- Provides strong consistency and high availability.

8 NODE FAILURE

- If a node goes down:
  - Clients should reconnect automatically.
  - Quorum queues elect a new leader.
  - Non-replicated queues on that node become unavailable.
  - Monitoring and alerts help you detect failures early.

9 NETWORK PARTITIONS

- Occurs when nodes cannot communicate due to network issues.
- Can lead to split-brain scenarios.
- Quorum queues require a majority of nodes to stay available.
- Use reliable networking, proper DNS and monitoring to prevent this.

10 CLIENT RECONNECTION

- Clients should be configured to automatically reconnect.
- Use connection recovery features in your client library.
- Handle failures with retries and backoff.
- Always use multiple node endpoints (or a load balancer).

11 LOAD BALANCERS

- Helps distribute client connections across nodes.
- Use a TCP load balancer (e.g. HAProxy, NLB).
- Should perform health checks.
- Do not use an HTTP load balancer for AMQP traffic.
- Ensure clients can still reconnect if a node becomes unavailable.

12 IMPORTANT REALITY CHECK

"Three nodes" does not automatically mean your application is highly available.

- You must also configure the right queue type (e.g. quorum queues).
- Ensure clients reconnect properly.
- Use load balancing and health checks.
- Monitor the cluster.
- Test failure scenarios.
- High availability requires correct configuration, not just more nodes.

JUNIOR ENGINEER TIP

RabbitMQ high availability is a combination of the right queue type, correct configuration, reliable networking, client reconnection, monitoring and regular testing. Infrastructure is more than just running multiple nodes.

STABLE MESSAGING BUILDS RELIABLE APPLICATIONS.

Learn Build Deploy Grow

RABBITMQ
SECURITY AND ACCESS CONTROL

Protect Your Messaging Infrastructure

1 AUTHENTICATION

- RabbitMQ authenticates users before they can connect.
- Built-in authentication backend (default).
- Supports external backends (LDAP, OAuth2, etc.).
- Use strong passwords or certificate-based authentication (TLS).

2 USERS

- Users are created in RabbitMQ.
- Each user has a password and permissions.
- Use separate users for different applications environments (dev, test, prod).
- Avoid using the default guest user.

3 VIRTUAL HOSTS

- Virtual hosts (vhosts) provide isolation.
- Each vhost has its own exchanges, queues, bindings and permissions.
- Use vhosts to separate teams, applications or environments.
- Example: / (default), /dev, /test, /prod.

4 PERMISSIONS

- Permissions control what a user can do on a vhost.
- Three types of permissions:
  - Configure: create and delete exchanges, queues, bindings.
  - Write: publish messages.
  - Read: consume messages.
- Set least privilege access based on application needs.

5 PERMISSION EXAMPLE

Permission	What It Allows
Configure	Create/delete exchanges, queues, bindings
Write	Publish messages to exchanges
Read	Consume messages from queues

6 LEAST PRIVILEGE

- Give only the permissions required.
- Do not use "administrator" access for applications.
- Use separate users per service.
- Review permissions regularly.
- Reduces risk if a credential is compromised.

7 TLS (ENCRYPTION)

- Use TLS to encrypt traffic between clients and RabbitMQ.
- Protects credentials and message data in transit.
- Configure TLS for AMQP (5671) and Management UI (HTTPS).
- Use trusted certificates (internal CA or public CA).

8 PROTECTING CREDENTIALS

- Do not hardcode credentials in application code.
- Use environment variables or configuration files.
- Restrict access to credential files.
- Use TLS to prevent credential sniffing.
- Follow your organization's secrets management policy.

9 SECRET MANAGERS

- Use a secret manager to store RabbitMQ credentials.
- Examples: HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, Google Secret Manager.
- Integrate with your application or CI/CD pipeline.
- Rotate credentials regularly.

10 ROTATING CREDENTIALS

- Change passwords regularly.
- Automate rotation using a secret manager.
- Update applications with minimal downtime.
- Revoke old credentials.
- Monitor for failed login attempts after rotation.

11 MANAGEMENT UI EXPOSURE

RabbitMQ Management UI runs on port 15672 (HTTP) or HTTPS.

- Do not expose it to the public internet.
- Restrict access to internal networks or VPN.
- Use strong authentication and TLS.
- Monitor access logs.

12 NETWORK RESTRICTIONS

- Restrict access to AMQP (5672/5671) and Management UI (15672/HTTPS).
- Use firewalls and security groups.
- Allow only trusted IP ranges (e.g. application servers, VPN).
- Avoid open access from the internet.
- Combine with TLS and strong authentication.

13 DEFAULT GUEST ACCOUNT RISKS

- The default guest user can only connect from localhost (127.0.0.1).
- Do not enable or expose the guest user in production.
- Attackers often target default credentials.
- Always create your own users with strong passwords.

14 SECURITY REMINDER

Never hardcode RabbitMQ credentials in source control.

Use a secret manager, environment variables or a secure configuration management tool.

Better Engineers A Stronger Tomorrow

VERIQTA

VERIQTA

LEARN | BUILD | DEPLOY | GROW

PAGE 16

Better Engineers A Stronger Tomorrow

RABBITMQ RESOURCE MANAGEMENT AND BACKPRESSURE

Keep Your Messaging System Healthy and Stable in Production

1 MEMORY USAGE

RabbitMQ uses memory for:

- Queues (especially classic queues)
- Message metadata
- Connections and channels
- Internal Erlang processes

- High memory usage can trigger a memory alarm and block publishers.

2 DISK USAGE

Disk is used for:

- Persisted messages
- Queue data (especially quorum queues)
- Message logs and metadata

- High disk usage can trigger a disk alarm and block publishers.

3 FILE DESCRIPTORS

- RabbitMQ uses file descriptors for:
- Network connections
- Open files (logs, queues, etc.)
- Each connection and channel consumes descriptors

- If you run out of file descriptors, new connections will fail.

4 MEMORY ALARMS

- Triggered when memory usage exceeds a configured threshold.
- RabbitMQ will block publishers using flow control.
- Helps protect the node from crashing.
- Messages are not lost, publishers are paused.

5 DISK ALARMS

- Triggered when disk usage exceeds a configured threshold.
- RabbitMQ will block publishers to prevent disk from being full.
- Protects the node and ensures message durability.
- You must free disk space or increase storage.

6 FLOW CONTROL

- When a node is under high memory or disk pressure, RabbitMQ enables flow control.
- Publishers are blocked (or slowed down) until the resource recovers.
- This prevents the node from crashing.
- Flow control is automatic and protects your data.

7 BACKPRESSURE

- Backpressure is the natural slowing down of publishers when the system is under pressure.
- It keeps RabbitMQ stable.
- Your application should handle backpressure and retries properly.
- Do not ignore blocked publishers in your application.

8 QUEUE GROWTH

- Queues grow when producers send messages faster than consumers can process them.
- Large queues consume memory and disk space.
- Monitor queue depth and consumer capacity.
- Uncontrolled growth can lead to alarms and blocked publishers.

9 CONSUMER CAPACITY

- Consumer capacity shows how fast consumers can process messages.
- If consumers are slow, queues will grow.
- Monitor consumer capacity in the management UI.
- Scale consumers or optimize processing if needed.

10 PREFETCH TUNING

- Prefetch (basic.qos) controls how many messages a consumer receives at a time.
- A lower value prevents a single consumer from being overloaded.
- A higher value can improve throughput but uses more memory.
- Tune based on your workload.

11 MESSAGE SIZE

- Large messages consume more memory and disk space.
- Keep messages small (ideally below a few MB).
- For large data, store it in external storage (e.g. S3) and send a reference in the message.
- Avoid sending binary files directly in RabbitMQ.

12 QUEUE LENGTH LIMITS

- You can set a maximum queue length (x-max-length) to prevent queues from growing indefinitely.
- When the limit is reached, messages can be dropped or dead-lettered.
- Helps protect the system from running out of memory or disk.

13 WHAT HAPPENS WHEN RABBITMQ PROTECTS ITSELF?

- When memory or disk alarms are triggered:
- RabbitMQ blocks publishers using flow control.
- Publishers will see blocked connections or slower publish rates.
- Consumers can continue to process existing messages.
- Once resources recover, publishing is automatically resumed.
- This behavior protects your data and keeps the node stable.

14 KEY TAKEAWAYS

- Monitor memory, disk and file descriptors.
- Set appropriate limits and alarms.
- Handle backpressure in your application.
- Keep messages small and consumers healthy.
- Use queue length limits and prefetch tuning.
- RabbitMQ will protect itself — design for it, not against it.

JUNIOR ENGINEER TIP

A healthy RabbitMQ system is not about pushing more messages, but about keeping the right balance between producers, consumers and resources.

REMEMBER

If you ignore resource limits, RabbitMQ will slow you down even if your application is sending messages successfully.

Better Engineers A Stronger Tomorrow

RABBITMQ
MONITORING RABBITMQ BEFORE USERS NOTICE

Monitor the Right Metrics. Detect Issues Early. Keep Your Systems Reliable.

1 QUEUE DEPTH

- Total number of messages in a queue.
- High queue depth can indicate slow consumers or stuck messages.
- Monitor per queue, not just the total.

- Alert when queue depth exceeds normal baseline.

2 READY MESSAGES

- Messages waiting to be delivered to consumers.
- Consistently high ready messages can indicate consumer issues.

- Set alerts for unusual growth in ready messages.

3 UNACKNOWLEDGED MESSAGES

- Messages delivered to consumers but not yet acknowledged.
- High unacked messages may indicate slow or failed consumers.

- Alert when unacked messages keep increasing.

4 PUBLISH RATE

- Number of messages published per second.
- Helps you understand traffic patterns and spikes.
- Monitor per exchange or virtual host.

5 DELIVER RATE

- Number of messages delivered to consumers per second.
- Compare with publish rate to detect bottlenecks.

6 ACKNOWLEDGEMENT RATE

- Number of messages acknowledged per second.
- Should normally be close to the deliver rate.
- A low ack rate may indicate consumer processing issues.

7 CONSUMER COUNT

- Total number of active consumers.
- A sudden drop can indicate consumer failures.
- Monitor per queue.

8 CONSUMER CAPACITY

- Shows how fast consumers can process messages (0 to 100%).
- Low consumer capacity means consumers are not keeping up.

9 CONNECTIONS

- Total number of client connections.
- Monitor for unexpected spikes or drops.
- Too many connections can exhaust resources.

10 CHANNELS

- AMQP channels per connection.
- Monitor for unusually high channel counts.
- Channel leaks can cause resource exhaustion.

11 MEMORY

- RabbitMQ uses memory for queues, connections, channels and metadata.
- High memory usage can trigger memory alarms.
- Monitor memory trends over time.

12 DISK

- Used for persistent messages, queue data and logs.
- High disk usage can trigger disk alarms and block publishers.
- Monitor disk space and growth rate.

13 NODE HEALTH

- Check node status, uptime and cluster health.
- Monitor for node failures, network issues and partitions.
- Ensure all nodes are running and communicating.

14 PROMETHEUS METRICS

- RabbitMQ exposes metrics for Prometheus.
- Monitor queues, connections, channels, memory, disk and more.
- Use the official RabbitMQ Prometheus exporter.

15 GRAFANA DASHBOARDS

- Visualize RabbitMQ metrics with Grafana.
- Use official or community dashboards.
- Create dashboards for queues, consumers, nodes and resources.
- Set up alerts with Grafana Alerting.

16 ALERTING ON SYMPTOMS

- Do not just monitor "RabbitMQ is up".
- Alert on symptoms such as high queue depth, high unacked messages, low consumer capacity, memory or disk alarms, and dropped connections.
- Set meaningful thresholds based on your environment.

17 JUNIOR ENGINEER TIP

Monitoring is not just about uptime.
It is about detecting problems before your users notice them.

Proactive Monitoring Prevents Outages

VERIQTA

VERIQTA

LEARN | BUILD | DEPLOY | GROW

PAGE 18

Better Engineers A Stronger Tomorrow

RABBITMQ

TROUBLESHOOTING PRODUCTION FAILURES

Find the Problem. Understand the Cause. Restore the Service.

1 MESSAGES ARE NOT REACHING A QUEUE

- Check exchange name and type.
- Verify routing key.
- Check bindings between exchange and queue.
- Ensure the queue exists.
- Look for unroutable messages (increase logging or use alternate exchange).

2 MESSAGES ARE STUCK IN A QUEUE

- Check if consumers are running.
- Verify consumer logs for errors.
- Look at unacknowledged messages.
- Check message TTL and DLX configuration.
- Ensure consumers can process the message format.

3 QUEUE DEPTH KEEPS INCREASING

- Consumers are slower than producers.
- Not enough consumers.
- Consumers failing or crashing.
- Messages being requeued.
- Check consumer capacity and prefetch.
- Look for downstream service issues.

4 CONSUMER DISAPPEARED

- Check if the consumer process is running.
- Look at application logs.
- Verify connection and channel are still open.
- Check for crashes or OOM kills (in Kubernates).
- Ensure auto-restart is configured.

5 CONNECTION REFUSED

- RabbitMQ service is down or unreachable.
- Check hostname, port (5672) and network connectivity.
- Verify firewall rules and security groups.
- Check if the node is accepting connections.
- Look at RabbitMQ logs.

6 AUTHENTICATION FAILURE

- Verify username and password.
- Check if the user exists.
- Ensure the user has access to the correct virtual host.
- Check for expired credentials.
- Verify TLS/certificate issues.
- Look at authentication logs in RabbitMQ.

7 UNROUTABLE MESSAGES

- Check routing key and exchange type.
- Verify queue bindings.
- Enable publisher confirms and mandatory flag.
- Use an alternate exchange to capture unroutable messages.

8 UNACKED MESSAGES INCREASING

- Consumers are not acknowledging messages.
- Consumers may be stuck or processing slowly.
- Check consumer logs and application performance.
- Verify prefetch configuration.
- Look for downstream service dependencies.

9 MEMORY ALARM

- RabbitMQ is using too much memory.
- Check for large messages.
- Reduce consumer prefetch if needed.
- Add more resources or scale consumers.
- Look for memory leaks in consumers.

10 DISK ALARM

- Disk space is low.
- Large queues or persistent messages.
- Clean up old messages or configure TTL.
- Increase disk space.
- Check logs for disk alarm events.

11 NODE UNAVAILABLE

- The node may be down or network partitioned.
- Check cluster status.
- Verify Erlang distribution between nodes.
- Check system resources (CPU, memory, disk).
- Review logs on the affected node.

12 USEFUL DIAGNOSTICS

```
rabbitmqctl status
rabbitmqctl cluster_status
rabbitmqctl list_queues
rabbitmqctl list_connections
rabbitmqctl list_channels
rabbitmqctl list_consumers
rabbitmq-diagnostics check_running
rabbitmq-diagnostics memory_breakdown
rabbitmq-diagnostics disk_free
rabbitmq-logs
```

13 TROUBLESHOOTING MODEL

Producer	Exchange	Binding	Queue	Consumer	Dependency
Is the message being published?	Is the routing key correct?	Is the binding in place?	Is the message in the queue?	Is the consumer processing it?	Is the downstream service healthy?

14 FIND WHERE THE MESSAGE JOURNEY BREAKS

- Follow the message step by step from producer to consumer.
- Verify each component and its logs.
- Use metrics, logs and RabbitMQ CLI.
- Check application and infrastructure dependencies.
- Test with a simple message.
- Do not assume the problem is in RabbitMQ.
- Identify the exact point of failure and fix it.

JUNIOR ENGINEER TIP

Always verify the full message journey. The issue is often one step before or after where you think it is.

REMEMBER

A methodical approach saves time. Check, verify, and isolate the problem before making changes.

Better Engineers A Stronger Tomorrow

RABBITMQ IN KUBERNETES AND PRODUCTION ARCHITECTURE

Run RabbitMQ Reliably at Scale in Kubernetes

1 RABBITMQ AS STATEFUL INFRASTRUCTURE

- RabbitMQ stores queue data, metadata and configuration.
- It requires persistent storage.
- It is stateful, not stateless.
- Treat it as critical infrastructure like a database.
- Design for durability, high availability and backups.

- Stateful workloads need careful planning, not just a deployment YAML.

2 STATEFULSETS

- Use a StatefulSet for RabbitMQ nodes.
- Provides stable network identities (pod-0, pod-1, pod-2).
- Maintains persistent storage per pod.
- Ensures ordered deployment, scaling and termination.
- Required for clustering.

- Do not use a Deployment for RabbitMQ in production.

3 PERSISTENT STORAGE

- Use PersistentVolumeClaims (PVC) for each node.
- Choose reliable storage (SSD recommended).
- Ensure enough disk space for messages and logs.
- Use a suitable StorageClass (e.g. gp3, premium SSD).
- Configure retention and monitor disk usage.

- Losing storage can lead to permanent message loss.

4 SERVICES

- Use a headless service for intra-cluster communication.
- Use a regular service for client access (ClusterIP, LoadBalancer or NodePort).
- Keep service configuration stable.
- Consider a load balancer for client connections.

5 DNS

- Kubernetes DNS provides stable hostnames for StatefulSet pods (e.g. rabbit-0.rabbit).
- Used for cluster communication.
- Allows clients to connect to a stable service name.
- Ensure proper DNS resolution across namespaces if needed.

6 RESOURCE REQUESTS AND LIMITS

- Set CPU and memory requests.
- Set appropriate limits.
- Avoid overcommitting resources.
- Monitor actual usage and adjust.
- Memory and disk limits are critical for stability.

- Under-resourced nodes can crash. Over-resourced nodes waste cluster capacity.

7 POD DISRUPTION CONSIDERATIONS

- Use PodDisruptionBudgets (PDB).
- Prevent too many nodes from being unavailable at the same time.
- Consider the impact of node drains, upgrades and maintenance.
- Test disruption scenarios.

- Do not allow all RabbitMQ pods to be evicted at once.

8 ANTI-AFFINITY

- Use pod anti-affinity to spread nodes across different nodes.
- Avoid running multiple RabbitMQ pods on the same Kubernetes node.
- Improves resilience and fault tolerance.

- If one node fails, multiple RabbitMQ pods should not be affected.

9 AVAILABILITY ZONES

- Deploy nodes across multiple availability zones.
- Protects against zone or datacenter failures.
- Ensure storage also supports multi-zone deployment.
- Test failover and recovery.

- High availability requires more than one zone.

10 RABBITMQ CLUSTER KUBERNETES OPERATOR

- Use the official RabbitMQ Cluster Kubernetes Operator.
- Simplifies deployment, scaling and management.
- Handles clustering, upgrades and configuration.
- Recommended for production.

11 SECRETS

- Store RabbitMQ credentials in Kubernetes Secrets.
- Do not use hardcoded credentials in manifests.
- Use external secret managers (e.g. HashiCorp Vault, AWS Secrets Manager) if possible.
- Rotate credentials regularly.

12 MONITORING

- Monitor with Prometheus and Grafana.
- Track key metrics: queue depth, node health, memory, disk, connections, channels and consumer activity.
- Set up alerts for critical thresholds.
- Use the management plugin for visual checks.

13 BACKUPS AND DEFINITIONS

- Regularly backup definitions (exchanges, queues, users, policies).
- Use rabbitmqctl or the management UI.
- Store backups in a safe external location.
- Test restore procedures.
- Also plan for persistent volume backups.

14 UPGRADES

- Plan upgrades carefully.
- Read the RabbitMQ and operator release notes.
- Test in a staging environment.
- Perform rolling upgrades.
- Verify cluster health after upgrade.
- Have a rollback plan.

15 KEY TAKEAWAY

Why restarting Pods blindly is not troubleshooting:

- It does not fix configuration issues.
- It can cause unnecessary downtime.
- It may hide the real problem.
- It can lead to message loss.
- Always investigate the root cause first.

JUNIOR ENGINEER TIP

Treat RabbitMQ in Kubernetes like a database. Plan for state, storage, monitoring, backups and failure scenarios. Stability comes from design, not luck.

REMEMBER

Restarting Pods is a last resort, not a troubleshooting method. Find the root cause, fix it, then verify the system is healthy.

Better Engineers
A Stronger Tomorrow

RABBITMQ
FINAL PRODUCTION LAB
BREAK AND RECOVER RABBITMQ

Hands-On Practice. Real Scenarios. Production Skills.

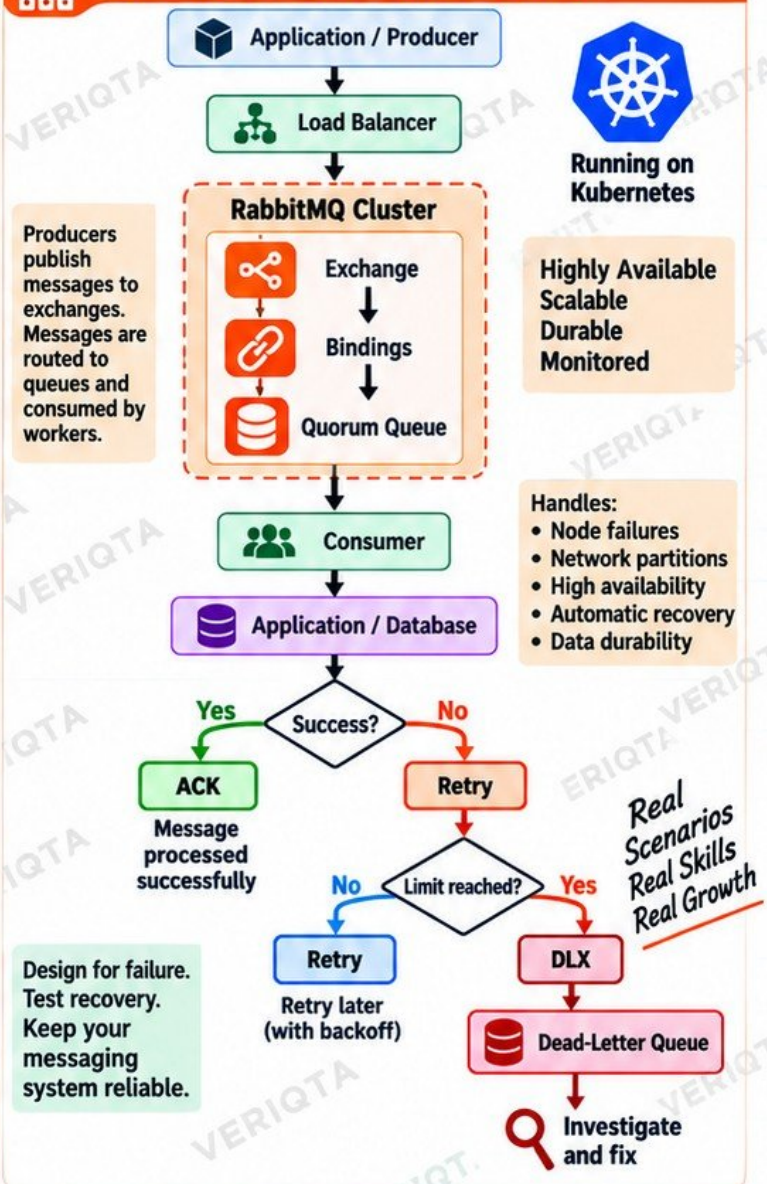
Practice
Break
Fix
Learn
Repeat

Same Knowledge
A Stronger You

LAB STEPS

1. Deploy a production-style RabbitMQ environment (standalone or Kubernetes).
2. Create users and permissions.
3. Create exchange, queues and bindings.
4. Publish messages.
5. Start consumers.
6. Enable manual acknowledgements.
7. Configure publisher confirms.
8. Configure dead lettering.
9. Add a controlled retry strategy.
10. Observe queue metrics.
11. Stop a consumer.
12. Watch queue depth increase.
13. Restart the consumer.
14. Introduce a poison message.
15. Trace it into the dead-letter path.
16. Simulate a node or application failure.
17. Diagnose the system.
18. Recover service.
19. Verify message processing.
20. Document what you learned.

FINAL PRODUCTION ARCHITECTURE



IMPORTANT REMINDER
Restarting Pods blindly is not troubleshooting.
Find the root cause, fix it, then verify the system is healthy.

THE FULL LEARNING JOURNEY

Produce → Route → Queue → Consume → Acknowledge → Retry → Dead-Letter → Monitor → Diagnose → Recover